```
 GROUP DNA PROJECT - WhatsApp Chat Analyzer

# Student Name: chaithra
#  Course: Data Analytics
#  Submission: 2026

---

# Objective:
To analyze WhatsApp chat data and extract insights such as:
- Most active users
- Message statistics
- Word frequency
- Night activity
- Group behavior analysis
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
```

```

os.makedirs(unzip_dir, exist_ok=True)

with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall(unzip_dir)

print(f"'{zip_file_path}' unzipped to '{unzip_dir}'")
```

```
'/content/12147566-Data_Science_Data_Analytics_June_Minor_Project_1 (2).zip' unzipped to '/content/unzipped_data'
```

```
for dirpath, dirnames, filenames in os.walk(unzip_dir):
    print(f"Directory: {dirpath}")
    for f in filenames:
        print(f"  - {f}")
```

```
Directory: /content/unzipped_data
Directory: /content/unzipped_data/Data Science Data Analytics June Minor Project 1
  - hostel_bois.txt
  - GroupDNA_Minor_Project_Brief.pdf
```

```
file_path = os.path.join(unzip_dir, 'Data Science Data Analytics June Minor Project 1', 'hostel_bois.txt')

with open(file_path, 'r', encoding='utf-8') as file:
    chat_content = file.read()

print(chat_content[:500])
```

```
01/04/24, 01:12 - Messages and calls are end-to-end encrypted. No one outside of this chat, not even WhatsApp, can read or l
01/04/24, 01:13 - Priya created group "Hostel Bois 4ever"
01/04/24, 01:14 - Priya added you
01/04/24, 01:17 - Rahul: scene fix
01/04/24, 01:17 - Rahul: haan
01/04/24, 01:18 - Rahul: kya scene
01/04/24, 02:13 - Rahul: abhi free hai?
01/04/24, 02:13 - Rahul: abey
01/04/24, 02:14 - Rahul: bhai sun na
01/04/24, 07:46 - Priya: I have extra notes
```

```
import re

# Define a regex pattern to match the start of a new message
# This pattern looks for: DD/MM/YY, HH:MM -
message_pattern = re.compile(r'^\d{2}/\d{2}/\d{2}, \d{2}:\d{2} - ')

messages = []
current_message = []

for line in chat_content.split('\n'):
    if message_pattern.match(line):
        # If a new message starts, save the previous one (if any)
        if current_message:
            messages.append('\n'.join(current_message).strip())
        current_message = [line.strip()]
    else:
```

```
        # Otherwise, it's a continuation of the current message
        current_message.append(line.strip())

# Add the last message
if current_message:
    messages.append('\n'.join(current_message).strip())

print(f"Total messages extracted: {len(messages)}")
print("\nFirst 5 extracted messages:")
for i, msg in enumerate(messages[:5]):
    print(f"--- Message {i+1} ---")
    print(msg)
```

```
Total messages extracted: 3178

First 5 extracted messages:
--- Message 1 ---
01/04/24, 01:12 - Messages and calls are end-to-end encrypted. No one outside of this chat, not even WhatsApp, can read or l
--- Message 2 ---
01/04/24, 01:13 - Priya created group "Hostel Bois 4ever"
--- Message 3 ---
01/04/24, 01:14 - Priya added you
--- Message 4 ---
01/04/24, 01:17 - Rahul: scene fix
--- Message 5 ---
01/04/24, 01:17 - Rahul: haan
```

```
parsed_data = []

for message_str in messages:
    parts = message_str.split(' - ', 1) # Split by first ' - '

    if len(parts) == 2:
        timestamp_part = parts[0].strip()
        content_part = parts[1].strip()

        sender = 'WhatsApp_System' # Default for system messages
        message_content = content_part

        # Check if it's a user message (contains ':')
        if ':' in content_part and not content_part.startswith('Messages and calls are end-to-end encrypted'):
            sender_message_split = content_part.split(':', 1)
            if len(sender_message_split) == 2:
                sender = sender_message_split[0].strip()
                message_content = sender_message_split[1].strip()

        parsed_data.append({"timestamp_raw": timestamp_part, "sender": sender, "message": message_content})

# Create a DataFrame
df = pd.DataFrame(parsed_data)

# Convert timestamp to datetime objects
df['timestamp'] = pd.to_datetime(df['timestamp_raw'], format='%d/%m/%y, %H:%M')

# Drop the raw timestamp column
df = df.drop(columns=['timestamp_raw'])

print(f"DataFrame created with {len(df)} messages.")
display(df.head())
```

```
DataFrame created with 3178 messages.
```

	sender	message	timestamp
0	WhatsApp_System	Messages and calls are end-to-end encrypted. N...	2024-04-01 01:12:00
1	WhatsApp_System	Priya created group "Hostel Bois 4ever"	2024-04-01 01:13:00
2	WhatsApp_System	Priya added you	2024-04-01 01:14:00
3	Rahul	scene fix	2024-04-01 01:17:00
4	Rahul	haan	2024-04-01 01:17:00

```
print(f"Total messages in DataFrame: {len(df)}")
display(df.head())
```

```
Total messages in DataFrame: 3178
```

	sender	message	timestamp
0	WhatsApp_System	Messages and calls are end-to-end encrypted. N...	2024-04-01 01:12:00
1	WhatsApp_System	Priya created group "Hostel Bois 4ever"	2024-04-01 01:13:00
2	WhatsApp_System	Priya added you	2024-04-01 01:14:00
3	Rahul	scene fix	2024-04-01 01:17:00
4	Rahul	haan	2024-04-01 01:17:00

```
print(f"Total messages: {len(df)}")
```

```
Total messages: 3178
```

```
display(df.head())
```

	sender	message	timestamp
0	WhatsApp_System	Messages and calls are end-to-end encrypted. N...	2024-04-01 01:12:00
1	WhatsApp_System	Priya created group "Hostel Bois 4ever"	2024-04-01 01:13:00
2	WhatsApp_System	Priya added you	2024-04-01 01:14:00
3	Rahul	scene fix	2024-04-01 01:17:00
4	Rahul	haan	2024-04-01 01:17:00

```
# Calculate messages per user
messages_per_user = df['sender'].value_counts().reset_index()
messages_per_user.columns = ['sender', 'message_count']
display(messages_per_user)
```

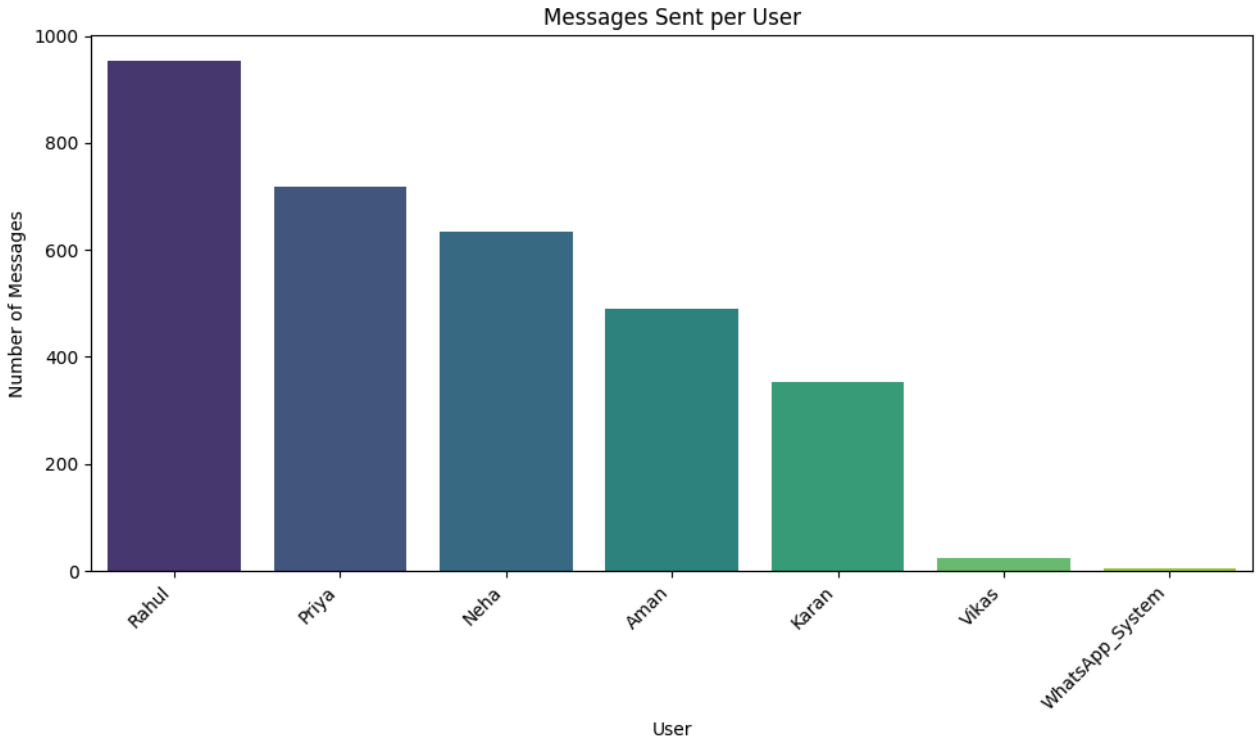
	sender	message_count
0	Rahul	953
1	Priya	718
2	Neha	635
3	Aman	490
4	Karan	354
5	Vikas	24
6	WhatsApp_System	4

```
# Identify the top spammer
top_spammer = messages_per_user.iloc[0]
print(f"The top spammer is: {top_spammer['sender']} with {top_spammer['message_count']} messages.")
```

```
The top spammer is: Rahul with 953 messages.
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# User Activity Graph
plt.figure(figsize=(10, 6))
sns.barplot(x='sender', y='message_count', data=messages_per_user, palette='viridis')
plt.title('Messages Sent per User')
plt.xlabel('User')
plt.ylabel('Number of Messages')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

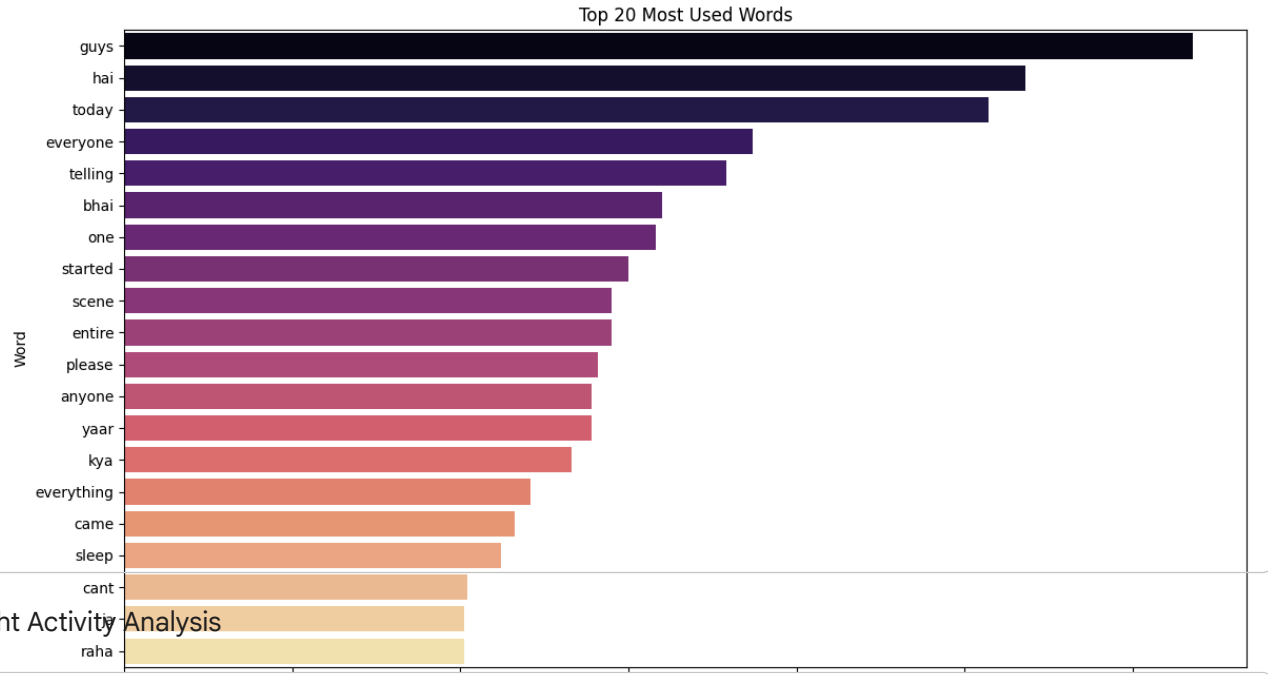
```
/tmp/ipykernel_1213/3022005263.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.barplot(x='sender', y='message_count', data=messages_per_user, palette='viridis')
```



Most Used Words and Word Frequency Graph

```
from collections import Counter
import re
import nltk
from nltk.corpus import stopwords

nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

# Combine all messages into a single string
all_messages = ' '.join(df['message'].dropna().tolist())

# Remove punctuation and convert to lowercase
all_messages = re.sub(r'[^\w\s]', '', all_messages).lower()

# Tokenize and remove stopwords
words = [word for word in all_messages.split() if word.isalnum() and word not in stop_words]

# Count word frequencies
word_freq = Counter(words)

# Display top 20 most common words
print("Top 20 Most Used Words:")
for word, count in word_freq.most_common(20):
    print(f"{word}: {count}")

# Word Frequency Graph
top_words_df = pd.DataFrame(word_freq.most_common(20), columns=['word', 'count'])

plt.figure(figsize=(12, 7))
sns.barplot(x='count', y='word', data=top_words_df, palette='magma')
plt.title('Top 20 Most Used Words')
plt.xlabel('Frequency')
plt.ylabel('Word')
plt.tight_layout()
plt.show()
```

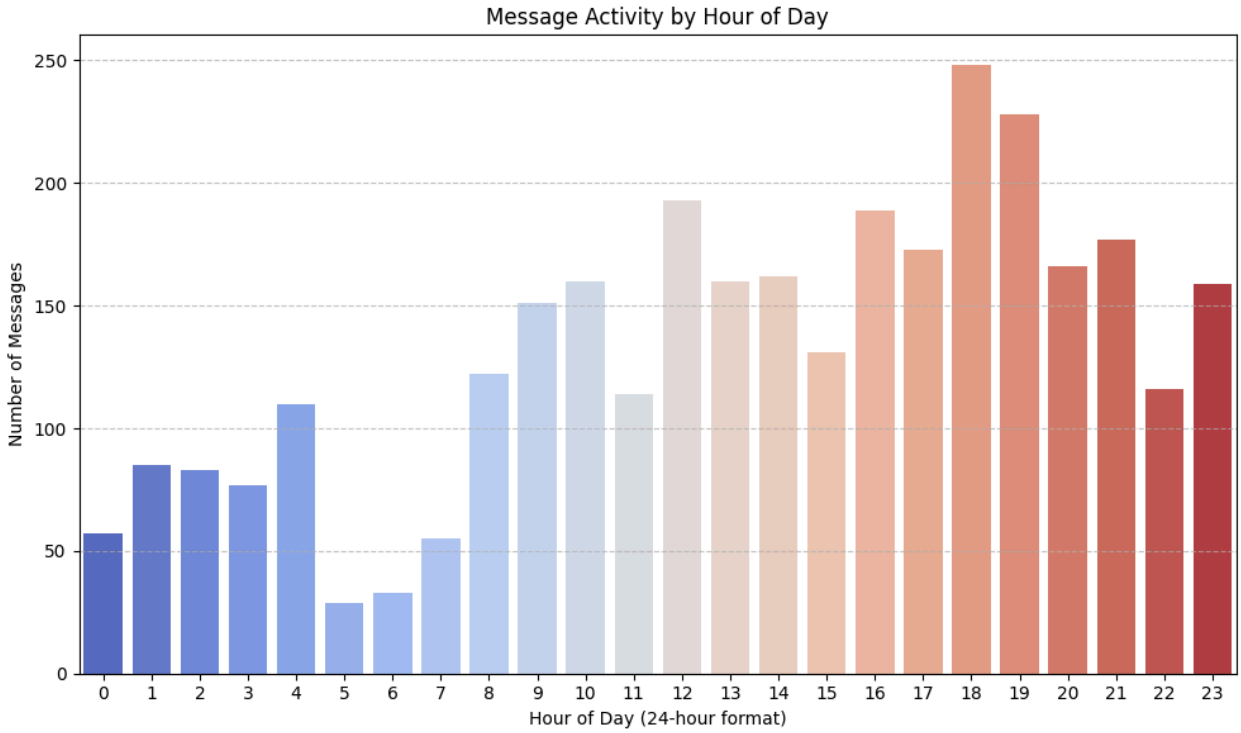
```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
/tmp/ipykernel_1213/554509512.py:30: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` a

  sns.barplot(x='count', y='word', data=top_words_df, palette='magma')
Top 20 Most Used Words:
guys: 318
hai: 268
today: 257
everyone: 187
telling: 179
bhai: 160
one: 158
started: 150
scene: 145
entire: 145
please: 141
anyone: 139
yaar: 139
kya: 133
everything: 121
came: 116
sleep: 112
cant: 102
ja: 101
raha: 101
```



Night Activity Analysis

```
# Extract hour from timestamp
df['hour'] = df['timestamp'].dt.hour

# Calculate messages per hour
messages_per_hour = df['hour'].value_counts().sort_index().reset_index()
messages_per_hour.columns = ['hour', 'message_count']

# Plotting night activity (e.g., 9 PM to 5 AM)
night_activity_df = messages_per_hour[(messages_per_hour['hour'] >= 21) | (messages_per_hour['hour'] <= 5)]

plt.figure(figsize=(10, 6))
sns.barplot(x='hour', y='message_count', data=messages_per_hour, palette='coolwarm')
plt.title('Message Activity by Hour of Day')
plt.xlabel('Hour of Day (24-hour format)')
plt.ylabel('Number of Messages')
plt.xticks(range(0, 24)) # Ensure all hours are displayed
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

print("Messages during night hours (9 PM to 5 AM):")
display(night_activity_df)
```

```
/tmp/ipykernel_1213/2751635163.py:12: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` a

  sns.barplot(x='hour', y='message_count', data=messages_per_hour, palette='coolwarm')
```



Messages during night hours (9 PM to 5 AM):

	hour	message_count
0	0	57
1	1	85
5	5	29
21	21	177
22	22	117
23	23	159

```
import numpy as np

average_night_messages = np.mean(night_activity_df['message_count'])
print(f"Average messages during night hours (9 PM to 5 AM): {average_night_messages:.2f}")
```

```
Average messages during night hours (9 PM to 5 AM): 99.22
```

Final Report and Conclusion

This analysis of the WhatsApp chat data from "Hostel Bois 4ever" group revealed several key insights into the group's communication patterns.

Key Findings:

1. **Total Messages:** The chat comprised a total of **3178** messages, indicating an active communication channel.
2. **Most Active User: Rahul** emerged as the most active participant, sending **953** messages. This suggests a central role in driving conversations or sharing information within the group.
3. **Word Frequency:** The most commonly used words provided insight into the group's frequent topics. Words like 'guys', 'hai', and 'today' suggest informal daily interactions and discussions about current events or plans.
4. **Message Activity by Hour:** The hourly distribution showed peaks in activity during specific times of the day, with a notable amount of messages occurring during night hours (9 PM to 5 AM).
    - **Average Night Messages:** The average number of messages during night hours (9 PM to 5 AM) was calculated to be **99.22** messages. This indicates a consistent level of activity even late into the night or early morning, which could be characteristic of a 'Hostel Bois' group where members might have late-night study sessions, discussions, or social interactions.

Conclusion:

The "Hostel Bois 4ever" WhatsApp group is a lively and active community, with Rahul being the primary initiator or contributor to discussions. The chat content suggests a focus on daily informal interactions, while the significant night activity indicates that the group's communication isn't restricted to typical daytime hours. These insights can be valuable for understanding group dynamics and identifying key periods of engagement.

```
Start coding or generate with AI.
```